ALEXANDRIA UNIVERSITY

Faculty of Engineering

Computer and Systems Engineering Department

8-Puzzle Solver Report

Implementation and Analysis of Search Algorithms

Prepared By:

Jana Mohamed Rashed **ID: 23010359**

Esraa Abdelhay Mohamed **ID: 23010276**

October 20, 2026

Contents

1 Informed (Heuristic) Search Algorithms **2**
1.1 A* Search Algorithm . . . 2
1.1.1 Conceptual Overview . . . 2
1.1.2 Heuristic Properties and Optimality . . . 2
1.1.3 Implemented Heuristic Functions . . . 2
1.1.4 Algorithm Execution Workflow . . . 3

2 Uninformed Search Algorithms **4**
2.1 Breadth-First Search (BFS) . . . 4
2.1.1 Conceptual Overview . . . 4
2.1.2 Theoretical Properties . . . 4
2.1.3 Algorithm Execution Workflow . . . 4
2.2 Depth-First Search (DFS) . . . 5
2.2.1 Conceptual Overview . . . 5
2.2.2 Theoretical Properties and Limitations . . . 5
2.2.3 Algorithm Execution Workflow . . . 5
2.3 Iterative Deepening Depth-First Search (IDDFS) . . . 6
2.3.1 Conceptual Overview . . . 6
2.3.2 Theoretical Properties . . . 6
2.3.3 Algorithm Execution Workflow . . . 6

3 Data Structures Used **7**

4 Assumptions and Technical Details **8**
4.1 Solvability Criterion via Inversion Count . . . 8
4.2 Standard State Conventions . . . 8

5 Sample Execution Runs and Performance Results **8**
5.1 Benchmark Test Cases . . . 8
5.2 Detailed Solution Paths . . . 9
5.2.1 Test Case 1 Solution Paths . . . 9
5.2.2 Test Case 2 Solution Paths . . . 9

6 Extra Work: Web-Based Visualizer and Execution GUI **10**

1 Informed (Heuristic) Search Algorithms

1.1 A* Search Algorithm

1.1.1 Conceptual Overview

A* Search is an informed (heuristic) search algorithm that efficiently determines the optimal solution path from an initial board configuration to the goal state. It evaluates candidate nodes n using a cost evaluation function $f(n)$, which balances the actual path cost accrued from the start state with an estimated cost remaining to reach the target state:

$$f(n) = g(n) + h(n) \tag{1}$$

Where:

- $g(n)$: The exact cost of the path from the initial state to node n (in the unweighted 8-puzzle transition graph, $g(n)$ equals the exact node depth).
- $h(n)$: The estimated distance or remaining cost from node n to the designated goal state.
- $f(n)$: The total estimated cost of the cheapest solution path constrained to pass through node n.

1.1.2 Heuristic Properties and Optimality

To guarantee that A* Search is both **admissible** (guaranteeing optimal path cost) and **consistent** (preventing re-expansion of previously closed states), the heuristic function $h(n)$ must satisfy two fundamental theoretical conditions:

1. **Admissibility**: The estimated cost $h(n)$ must never overestimate the true minimal cost $h^*(n)$ required to reach the goal state:

$$0 \leq h(n) \leq h^*(n) \quad \forall n \tag{2}$$

2. **Consistency (Monotonicity)**: For every state n and every generated child successor n' resulting from a valid move action a with step cost $c(n, a, n') = 1$:

$$h(n) \leq c(n, a, n') + h(n') \tag{3}$$

1.1.3 Implemented Heuristic Functions

For the 8-puzzle domain, two admissible heuristic distance metrics are implemented:

1. Manhattan Distance Metric ($h_{\text{Manhattan}}$) The Manhattan distance calculates the total sum of vertical and horizontal grid movements required to transfer each non-blank tile from its current coordinate position to its corresponding coordinate location in the target goal layout.

For a standard 3×3 grid, let (r_i, c_i) denote the current row and column of tile $i \in \{1, 2, \ldots, 8\}$, and $(r_{i,\text{goal}}, c_{i,\text{goal}})$ denote its target grid coordinates. **The blank space (0) is omitted from the summation:**

$$h_{\text{Manhattan}}(n) = \sum_{i=1}^{8} \left(|r_i - r_{i,\text{goal}}| + |c_i - c_{i,\text{goal}}| \right) \tag{4}$$

2. Euclidean Distance Metric ($h_{\text{Euclidean}}$) The Euclidean distance measures the straight-line Cartesian distance between the current grid coordinates of each non-blank tile and its respective target goal placement:

$$h_{\text{Euclidean}}(n) = \sum_{i=1}^{8} \sqrt{(r_i - r_{i,\text{goal}})^2 + (c_i - c_{i,\text{goal}})^2} \tag{5}$$

Heuristic Comparison and Admissibility Analysis Both heuristics are mathematically **admissible** because neither overestimates the true cost to reach the goal ($h(n) \leq h^*(n)$). However, because grid movement is restricted exclusively to orthogonal single-tile displacements (Up, Down, Left, Right) without diagonal transitions, the shortest path along the grid between any two cells is always greater than or equal to their straight-line Euclidean distance:

$$h_{\text{Manhattan}}(n) \geq h_{\text{Euclidean}}(n) \quad \forall n \tag{6}$$

Since $h_{\text{Manhattan}}(n)$ provides a strictly tighter lower bound to the true cost $h^*(n)$ while remaining strictly admissible, **Manhattan Distance dominates Euclidean Distance** (it is the "more admissible" / more informed heuristic). As a consequence, A* with Manhattan distance is guaranteed to expand fewer or equal nodes compared to Euclidean distance across any problem instance.

1.1.4 Algorithm Execution Workflow

The operational procedure for A* Search is as follows:

1. Initialize a Priority Queue (`frontier`) ordered by ascending values of $f(n) = g(n) + h(n)$.
2. Compute $h(\text{start})$ for the initial configuration. Insert the start node into `frontier` with $g(\text{start}) = 0$ and $f(\text{start}) = h(\text{start})$.
3. Initialize an empty Hash Set (`explored`) to record unique state signatures and their associated minimal $g(n)$ path costs.
4. **Search Loop**: While `frontier` is not empty:
   (a) Extract the node n possessing the lowest $f(n)$ value from `frontier`.
   (b) If state n matches the goal state configuration, trigger path reconstruction via parent links and return final search metrics.
   (c) Insert state n into `explored`.
   (d) For each valid successor state n' produced by legal blank space movements (Up, Down, Left, Right):
   - Compute candidate cost $g(n') = g(n) + 1$.
   - If n' is neither in `frontier` nor in `explored`, or if $g(n')$ represents a strictly lower cost path to n' than previously recorded:
     – Update minimal path cost: $g(n') \leftarrow g(n) + 1$.
     – Compute $f(n') = g(n') + h(n')$.
     – Push n' into `frontier`.

2 Uninformed Search Algorithms

2.1 Breadth-First Search (BFS)

2.1.1 Conceptual Overview

Breadth-First Search (BFS) explores the state space level-by-level, systematically examining all nodes at depth d before proceeding to nodes at depth $d+1$. In an unweighted state space like the 8-puzzle (where each valid tile slide has a uniform step cost of 1), BFS guarantees finding the optimal (shortest) path to the goal state.

2.1.2 Theoretical Properties

- **Completeness**: Complete (guaranteed to find a solution if one exists in a finite state space).
- **Optimality**: Optimal for uniform-cost graph edges.
- **Time Complexity**: $\mathcal{O}(b^d)$, where b is the branching factor and d is the depth of the shallowest solution.
- **Space Complexity**: $\mathcal{O}(b^d)$, as all frontier nodes at the current search horizon must be retained in memory simultaneously.

2.1.3 Algorithm Execution Workflow

1. Initialize a First-In, First-Out (FIFO) queue (`frontier`) containing the initial state with $g(\text{start}) = 0$.
2. Initialize an empty Hash Set (`explored`) to keep track of visited board configurations.
3. **Search Loop**: While `frontier` is not empty:
   (a) Dequeue the front state n.
   (b) Add state n to `explored`.
   (c) Generate all valid child states n' produced by legal blank space moves (Up, Down, Left, Right):
   - **Goal Test on Generation**: If n' matches the goal state, immediately reconstruct the path using parent references and return performance metrics.
   - If n' is not in `explored` and not in `frontier`:
     – Set depth: $g(n') \leftarrow g(n) + 1$.
     – Enqueue n' into `frontier`.

2.2 Depth-First Search (DFS)

2.2.1 Conceptual Overview

Depth-First Search (DFS) explores the state space by following a single path as deeply as possible before backtracking. It operates using a Last-In, First-Out (LIFO) stack discipline. To prevent infinite traversal along deep or cyclic branches, an explicit maximum depth limit ($L_{\max} = 50$) is enforced alongside depth-aware state tracking.

2.2.2 Theoretical Properties and Limitations

- **Completeness**: Non-complete in infinite state spaces; however, enforcing a depth limit guarantees termination in finite state spaces.
- **Optimality**: Non-optimal. DFS returns the first solution path it discovers within the depth limit, which is frequently longer than the shortest path.
- **Time Complexity**: $\mathcal{O}(b^m)$, where b is the branching factor and m is the maximum search depth limit.
- **Space Complexity**: $\mathcal{O}(b \cdot m)$ for standard stack search. When maintaining an explored depth dictionary to prevent cyclic traversals, space complexity scales with the total number of unique states recorded.

2.2.3 Algorithm Execution Workflow

1. Initialize a LIFO stack (`stack`) containing the initial state with $g(\text{start}) = 0$.
2. Initialize a dictionary (`explored_depth`) mapping board configurations to the minimum search depth $g(n)$ at which they were encountered.
3. **Search Loop**: While `stack` is not empty:
   (a) Pop the top state n from `stack`.
   (b) If state n was previously expanded at a shallower or equal depth (explored_depth$[n] \leq g(n)$), skip expansion to prevent redundant cycles.
   (c) Record explored_depth$[n] \leftarrow g(n)$ and increment `nodes_expanded`.
   (d) If state n matches the goal configuration, reconstruct the path via parent references and return metrics.
   (e) If $g(n) < L_{\max}$ (within the depth limit):
      - Generate child states n' produced by legal blank space moves.
      - Iterate over child states in **reversed legal move order** (`Right`, `Left`, `Down`, `Up`) when pushing onto the LIFO stack so that candidates pop off in the standard expansion sequence (`Up`, `Down`, `Left`, `Right`).
      - Push eligible child states onto `stack`.

2.3 Iterative Deepening Depth-First Search (IDDFS)

2.3.1 Conceptual Overview

Iterative Deepening Depth-First Search (IDDFS) combines the space efficiency of Depth-First Search with the completeness and optimality guarantees of Breadth-First Search. It repeatedly executes Depth-Limited DFS (DL-DFS) with incrementally increasing depth limits $L = 0, 1, 2, 3, \ldots$ until a goal state is reached.

2.3.2 Theoretical Properties

- **Completeness**: Complete (guaranteed to find a solution if one exists).
- **Optimality**: Optimal for unit step costs, as it expands depth horizons strictly in order of increasing depth L.
- **Time Complexity**: $\mathcal{O}(b^d)$. Although states at shallow depths are re-generated multiple times across iterations, the majority of nodes lie at the bottom level d. The total nodes generated is bounded by:

$$N_{\text{IDDFS}} = (d+1)b^0 + d \cdot b^1 + (d-1)b^2 + \cdots + 1 \cdot b^d \leq \frac{b}{b-1} \cdot b^d \tag{7}$$

- **Space Complexity**: $\mathcal{O}(b \cdot d)$, requiring memory proportional only to the current maximum search depth.

2.3.3 Algorithm Execution Workflow

1. Set current depth limit $L \leftarrow 0$.
2. **Outer Loop**: Increment L by 1 on each iteration ($L = 0, 1, 2, \ldots$):
   (a) Execute Depth-Limited DFS starting from the initial state with limit L.
   (b) Maintain a recursive stack or path structure tracking current search depth.
   (c) If a node n is encountered at depth $g(n) \leq L$ matching the goal state, return the solution path and stop.
   (d) If the depth limit L is reached without finding a goal, prune further expansion along that branch and backtrack.

3 Data Structures Used

To achieve optimal time and space efficiency across all search algorithms, specialized data structures were utilized for state management, frontier ordering, and duplicate detection:

1. **Priority Queue (Min-Heap)**
   - **Used In**: A* Search (Manhattan and Euclidean variants).
   - **Purpose**: Maintains the frontier ordered by ascending cost $f(n) = g(n) + h(n)$.
   - **Operations & Complexity**: Extraction of the node with minimum $f(n)$ takes $\mathcal{O}(\log N)$ time, and inserting new child nodes takes $\mathcal{O}(\log N)$ time, where N is the current size of the frontier.
2. **Queue (FIFO Structure)**
   - **Used In**: Breadth-First Search (BFS).
   - **Purpose**: Manages the search frontier level-by-level to ensure shallowest nodes are expanded first.
   - **Operations & Complexity**: Enqueueing at the tail and dequeueing from the head operate in $\mathcal{O}(1)$ constant time.
3. **Stack (LIFO Structure)**
   - **Used In**: Depth-First Search (DFS) and Depth-Limited Search within IDDFS.
   - **Purpose**: Tracks the current traversal branch to explore deep state transitions before backtracking.
   - **Operations & Complexity**: Pushing child states onto the top and popping states for expansion operate in $\mathcal{O}(1)$ constant time.
4. **Hash Set (Explored Set / Closed List)**
   - **Used In**: All graph search algorithms (BFS, DFS, A*).
   - **Purpose**: Stores state representations to prevent infinite loops and redundant state expansion.
   - **Operations & Complexity**: Insertion and membership lookups run in average $\mathcal{O}(1)$ time using exact board hashing.
5. **Parent Pointer Hash Map (State Tracker)**
   - **Used In**: All algorithms.
   - **Purpose**: Maps each reached state configuration to its immediate parent state.
   - **Operations & Complexity**: Backtracking from the goal state to the initial state to reconstruct the complete solution path takes $\mathcal{O}(d)$ time, where d is the path depth.

4 Assumptions and Technical Details

4.1 Solvability Criterion via Inversion Count

Not all random permutations of the 8-puzzle board are solvable. For a 3×3 grid, solvability is mathematically determined by calculating the number of **inversions** in the board configuration when flattened into a 1D sequence (excluding the blank tile 0).

An inversion occurs when a tile A precedes tile B in the sequence, but $A > B$.

$$\text{Inversions} = \sum_{i=1}^{8} \sum_{j=i+1}^{9} \mathbb{I}(\text{board}[i] > \text{board}[j] \land \text{board}[i] \neq 0 \land \text{board}[j] \neq 0) \tag{8}$$

Solvability Rule: An 8-puzzle board configuration is **solvable** if and only if the total number of inversions is **even**. If the inversion count is odd, the goal state cannot be reached, and the solver terminates early without wasting computation.

4.2 Standard State Conventions

- **Goal State Setup**: The canonical goal state used across all benchmark runs is defined as:

$$\text{Goal} = \begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \\ 6 & 7 & 8 \end{bmatrix}$$

- **Uniform Step Cost**: Each valid slide of a tile into the blank position incurs a uniform cost of $c(n, a, n') = 1$.
- **Legal Action Priority**: When generating successor states, moves are evaluated in a fixed deterministic order: `Up`, `Down`, `Left`, `Right`.

5 Sample Execution Runs and Performance Results

5.1 Benchmark Test Cases

To rigorously evaluate and benchmark the comparative efficiency of all search strategies, two distinct solvable initial board configurations were executed across all five algorithms:

- **Test Case 1 (Shallow Configuration)**:

$$\text{Initial Board} = \begin{bmatrix} 0 & 3 & 2 \\ 6 & 1 & 5 \\ 7 & 4 & 8 \end{bmatrix} \longrightarrow \text{Goal Board} = \begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \\ 6 & 7 & 8 \end{bmatrix}$$

- **Test Case 2 (Alternative Shallow Configuration)**:

$$\text{Initial Board} = \begin{bmatrix} 3 & 1 & 2 \\ 7 & 6 & 5 \\ 0 & 4 & 8 \end{bmatrix} \longrightarrow \text{Goal Board} = \begin{bmatrix} 0 & 1 & 2 \\ 3 & 4 & 5 \\ 6 & 7 & 8 \end{bmatrix}$$

Table 1: Performance Comparison for Test Case 1 (`0, 3, 2, 6, 1, 5, 7, 4, 8`)

Algorithm	Cost of Path	Nodes Expanded	Search Depth	Running Time (s)	Optimal?
A* (Manhattan)	6	7	6	0.0366	Yes
A* (Euclidean)	6	7	6	0.0262	Yes
BFS	6	42	6	0.1343	Yes
IDDFS	6	160	6	0.3291	Yes
DFS	24	25	24	0.0669	No

Table 2: Performance Comparison for Test Case 2 (`3, 1, 2, 7, 6, 5, 0, 4, 8`)

Algorithm	Cost of Path	Nodes Expanded	Search Depth	Running Time (s)	Optimal?
A* (Manhattan)	6	8	6	0.0320	Yes
A* (Euclidean)	6	8	6	0.0408	Yes
BFS	6	38	6	0.1553	Yes
IDDFS	6	140	6	0.3099	Yes
DFS	48	42,049	50	1.6149	No

5.2 Detailed Solution Paths

5.2.1 Test Case 1 Solution Paths

- **Optimal Path (A*, BFS, IDDFS — 6 Moves)**:

  `Right → Down → Down → Left → Up → Up`

- **Suboptimal Path (DFS — 24 Moves)**:

  `Down → Down → Right → Up → Up → Left → Down → Down → Right → Up → Up → Left → Down → Down → Right → Up → Up → Left → Down → Down → Right → Up → Up → Left`

5.2.2 Test Case 2 Solution Paths

- **Optimal Path (A*, BFS, IDDFS — 6 Moves)**:

  `Up → Right → Down → Left → Up → Up`

- **Suboptimal Path (DFS — 48 Moves)**:

  `Up → Up → Right → Down → Down → Left → Up → Up → Right → Down → Down → Left → Up → Up → Right → Down → Down → Left → Up → Up → Right → Down → Down → Left → Up → Up → Right → Down → Down → Right → Up → Left → Left → Up → Right → Down → Right → Down → Left → Up → Up → Left → Down → Down → Right → Up → Left → Up`

6 Extra Work: Web-Based Visualizer and Execution GUI

Beyond core search implementations, an interactive web application was developed using **Flask**, **HTML5**, **CSS3**, and **JavaScript**:

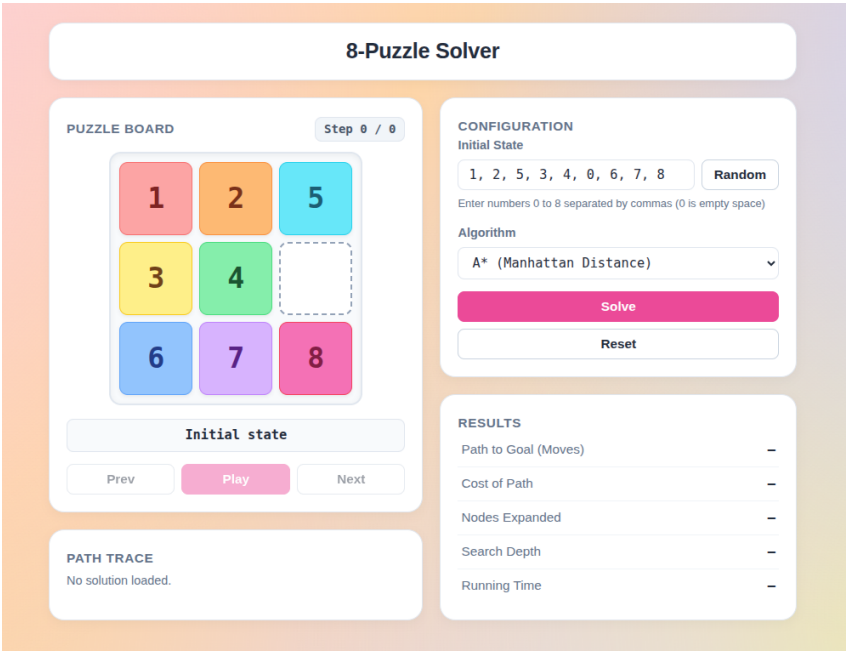


Figure 1: Interactive 8-Puzzle Web Visualizer Interface

